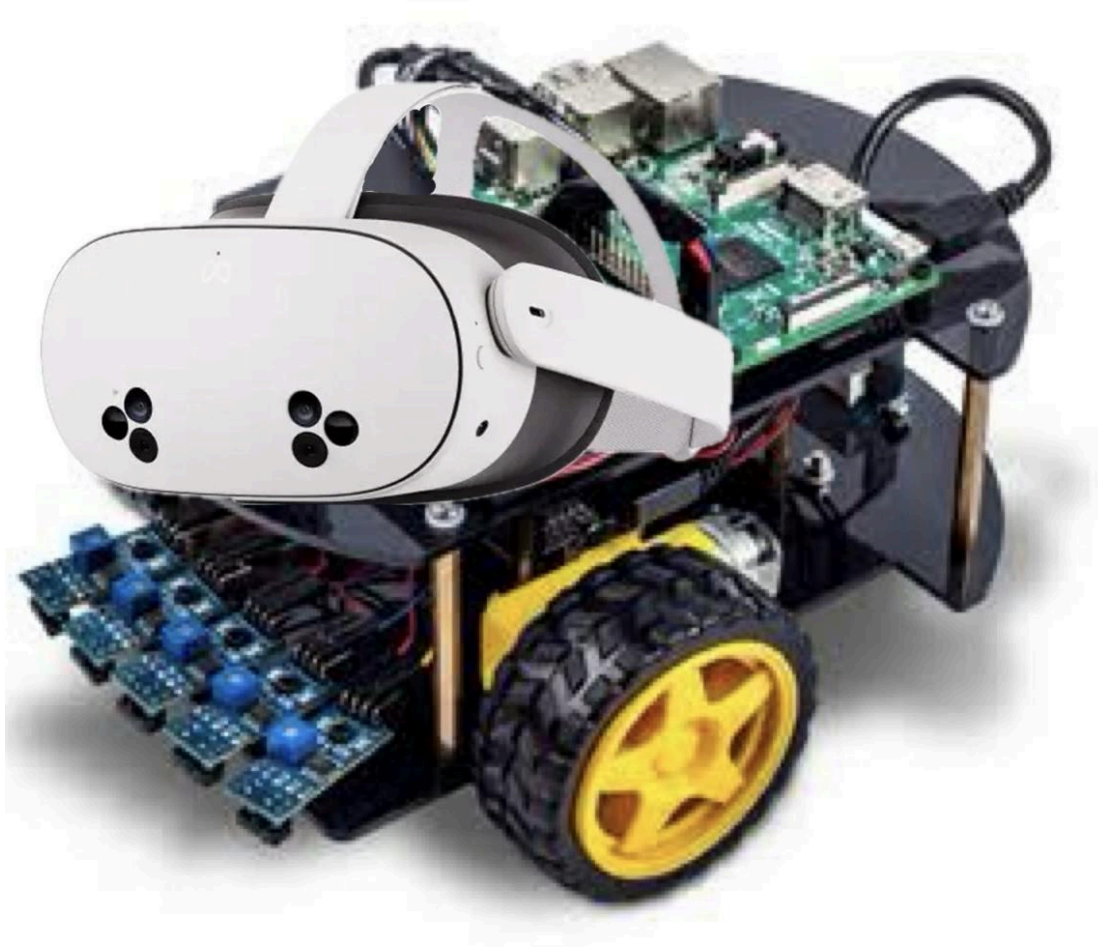


Logo (Matthew Palmer)



VIRTUAL ROBOTICS

Virtual Robotics Requirements Doc

Prepared by Wesley Burcham, Matthew Palmer, and Michael Brown

Introduction (Wesley Burcham):

In robotics, the question of how to control a robot has many answers. Because many small-scale robots have very similar interfaces that control them, in this project we're

exploring the idea of having one controller for many different kinds of robots that are able to travel in 3D space. Specifically, we've been tasked with implementing a robotic control suite in the Meta Quest VR platform that will allow us to send inputs to a microcontroller and coordinate motor movement, with the hopes of using this to create some version of a universal robotic controller.

Objective(s) (Matthew Palmer):

The user will be able to interact with a microcontroller connected to motors; ideally, this will be in the form of a vehicle, but it is not a must. The user will be able to see through the camera attached to the microcontroller and control the movement of the motors using the joysticks that come with the Meta Quest. The system will be developed on the Meta Quest but can possibly be adapted to other VR platforms like the Apple Vision Pro in future projects.

Functional Requirements (Michael Brown):

- (Must) The Meta Quest should be able to control a group of motors attached to a microcontroller. If put on a robot/drone with wheels, these motors would allow that robot/drone to move forwards, backwards, left, and right, although whether that comes to fruition is outside the control of this particular group. The deliverable in this project is the ability to interact with a microcontroller with a VR platform.
- (Must) The Meta Quest should be able to display the camera feed from the camera attached to the microcontroller.
- (Should) Track data/metrics about the robot related to how it's performing and where it is in space
 - Orientation (via the IMU)
 - Speed
 - Altitude (reference being sea level)

Non-Functional Requirements (Wesley Burcham):

Hardware Constraints:

The platform will be developed for the Meta Quest 3, and microcontroller interaction will primarily come from the joystick controllers that come with the Meta Quest. If someone wishes to use this application for other VR platforms, significant modifications will need to be made to multiple parts of the implementation, including but not limited to:

- The messaging between the VR and the microcontroller
- The controls themselves (the Apple Vision Pro does not come with joystick controllers)
- The application overall will need to be adapted to "fit" into Apple's platform . . . I do not recommend it unless you have the time.

Usability:

- The controls themselves will be simple; one joystick will act as the throttle, moving the motors in a way that will correlate to forward and backwards if/when

they're attached to the robot, and the other will act as the rotator, controlling the motors in such a way that they'll turn the robot when attached.

- Any data, including the camera feed, displayed will be made as visually appealing and intuitive as possible.

Performance:

- Development and test runs will take place on an isolated network on which speed and performance should be smooth and responsive. When taken off, performance will likely, but not certainly, be affected by network congestion.

- **Data Flow Outline:**

- **Controls:**

Quest controller joystick / hand pose
↓ WebXR Gamepad API
JavaScript in the Quest browser
↓ WebSocket over WiFi
Router
↓ WiFi
Pi 4 — Python server
↓ translate to motor commands
Pi 4 I2C bus
↓
PWM HAT (PCA9685)
↓ steady pulse widths
Model X driver board (H-bridge)
↓ switched battery voltage
Gear motors → wheels

- **Video:**

CSI camera sensor
↓ ribbon cable
Pi 4 image processor (ISP)
↓ on-chip
Pi 4 hardware H.264 encoder
↓ compressed frames
Pi 4 network stack
↓ WiFi 5GHz
TP-Link router
↓ WiFi 5GHz
Quest 3 browser
↓ decode
Panel floating in the VR view

Accessibility:

- The camera feed likely won't be adapted, but any graphics made for sensor/data display, if possible, will be in color blind friendly formats.

Risk Analysis (Matthew Palmer):

- A user might experience various kinds of disorientation when wearing the headset, such as motion sickness or epileptic seizures, and/or may trip over or bump into various real world objects if they don't observe their real world surroundings frequently while wearing the headset.

SPRINTS (All):

////////////////////////////////////

Sprint 1: Planning & Material Gathering:

- Create GitHub Repository
- Settle on what microcontroller will be used (likely Raspberry Pi)
- Gather/order materials for the robot:
 - 5GHz router (reach out to Dr. Hall to order)
 - NEED TO REACH OUT TO IT TO MAKE SURE IT'S ALLOWED)
 - Raspberry Pi 4 (3 and 2 for backup)
 - Ethernet cable
 - We've obtained a Raspberry Pi-controlled car. We need to investigate what it does and doesn't have that we need/don't need. If we end up not using it, we need the following:
 - Raspberry Pi Camera (reach out to Dr. Hall to order)
 - Arduino Nano
 - Meta Quest 3 (w/ controllers)
 - 2 motors
- Settle on which platform we're going to be programming with: Apple Vision Pro or Meta Quest

***ALL FOLLOWING SPRINTS ARE PREDICTED/IDEAL TIMELINES, ALL ARE SUBJECT TO CHANGE**

Sprint 2: Planning pt. 2 & Setup:

- Have everything we need to begin programming (Robot, Meta Quest, Router)
 - We are still waiting for parts such as batteries for our robot to arrive before we can do anything as of this Sprint.
- Setup Router and figure out/test connections
 - Meta Quest $\longleftrightarrow$ Router
 - Router $\longleftrightarrow$ Pi Car
- Start thinking about possible tests (tests performed may change based on factors such as condition of teammates or total progress made in our project)
 - Controls (may have to be physical tests rather than hardcoded):
 - Left Joystick (Throttle)

- Forward works
 - Backward Works
 - Right Joystick (Yaw/Direction)
 - Turning right works
 - Turning left works
 - Display:
 - Camera
 - feed is displaying
- Take note of what resources the platform has for programming it and draft a plan for translating our input into robot actions/movements.

Sprint 3: Implementation: Controls: Forward and Backwards Movement

- The goal of this sprint will be to get controls working, focusing on the left joystick to get forward and backwards motion working and implement a throttle (if the joystick is further the robot moves faster (if possible))

Sprint 4: Implementation: Controls: Yaw/Turning

- The goal of this sprint will be to get rotation working, focusing on the right joystick, ensuring that we can turn right and left. Additionally, we'll be making sure that this cooperates with the previous implementation and is able to smoothly transition between turning and accelerating.

Sprint 5: Implementation: Camera Feed: Functional Display

- The goal of this sprint will be to establish a camera feed and ensure that it's as smooth as possible especially when the robot is moving at higher relative speeds. This could involve capping the speed at which the robot can move, implementing an algorithm to smooth out the feed, or both.

Sprint 6: Implementation: Camera Feed: Glamorizing

- The goal of this sprint will be to clean up the camera feed, touching up any aspect that is uncomfortable for the user wearing the headset.
- (Optional): Add sensor display
 - Speed (assuming there is a sensor that can read such information)
 - Distance from user (to ensure that it stays within optimal distance to avoid disconnecting)
 - Distance if from objects in front (if possible with provided sensors)

Sprint 7: Implementation: Finalization/Grace Period

